

나만의 용돈 기입장 프로그램

학습 문제집

Codyyssey Subject 2 · B2-1 — Study.md 기반

이 문제집의 목적

- 과제 명세가 요구하는 "스스로 설명할 수 있어야 하는 5가지"를 반복해서 확인한다
- 확정된 설계 결정 8가지를 근거까지 함께 기억한다
- 구현 중 실제로 걸릴 함정을 미리 한 번씩 밟아본다

구성

- Part 1 — 빈칸 채우기 28문항
- Part 2 — 객관식 5지선다 22문항
- Part 3 — 코드 빈칸 채우기 5문항
- Part 4 — 서술형 12문항
- Part 5 — 코드 진단 3문항
- 정답 및 해설

푸는 법

- 기본 은 한 번에 답이 나와야 하는 문항, 심화 는 근거까지 말할 수 있어야 하는 문항이다
- 중요한 개념(제너레이터 · 계층 분리 · 데코레이터 · 원자적 쓰기)은 여러 Part에 걸쳐 반복해서 나온다
- 객관식은 답만 고르지 말고 "이유:" 칸을 반드시 채울 것 — 틀린 선택지가 왜 틀렸는지가 진짜 학습 지점이다
- 서술형은 키워드 나열이 아니라 완성된 문장으로 쓴다. 과제 목표가 "설명할 수 있는가"이기 때문이다

Part 1. 빈칸 채우기

문장을 읽고 빈칸에 들어갈 말을 쓰시오. 번호가 여러 개면 순서대로 답한다.

1-01 기본

저장 포맷은 한 줄에 JSON 객체 하나씩 담는 ⁽¹⁾ 형식을 선택했다. 파일 확장자는 `.jsonl` 이다.

1-02 기본

`tags: list[str] = []` 라고 쓰면 모든 인스턴스가 같은 리스트 객체를 ⁽¹⁾ 하는 버그가 난다. 올바른 표기는 `field( (2) =list)` 이다.

1-03 기본

`yield` 는 값을 하나 내보내고 함수 실행을 ⁽¹⁾ 한다. 함수를 완전히 끝내는 `return` 과 다른 지점이다.

1-04 기본

`update / delete` 는 파일 전체를 재작성하므로, 임시 파일에 먼저 쓰고 `os. (1) ( )` 로 교체해 ⁽²⁾ 을(를) 보장한다.

1-05 기본

데코레이터를 만들 때 원본 함수의 `__name__` 과 docstring을 유지하려면 `functools. (1)` 를 붙인다.

1-06 기본

정상 종료 시 exit code는 ⁽¹⁾ , 오류 종료 시에는 ⁽²⁾ 이 아닌 값이어야 한다.

1-07 심화

`-from` 옵션은 `from` 이 파이썬 ⁽¹⁾ 이기 때문에 `args.from` 으로 접근할 수 없다. `add_argument` 에 ⁽²⁾ 인자를 따로 지정해야 한다.

1-08 심화

초기 카테고리리는 파일이 ⁽¹⁾ 때만 시드한다. 파일이 ⁽²⁾ 때 시드하면 사용자가 전부 지운 뒤 재실행할 때 기본값이 되살아난다.

1-09 심화

CSV 파일을 열 때 `open(..., newline= (1) )` 를 주지 않으면 레코드 사이에 ⁽²⁾ 이(가) 삽입된다.

1-10 기본

한글이 `\uXXXX` 로 이스케이프되지 않게 하려면 `json.dumps(..., ensure_ascii=`⁽¹⁾`)` 로 저장한다.

1-11 심화

import/export CSV 최소 스키마에는 ⁽¹⁾ 컬럼이 없다. 그래서 export → import 왕복은 항상 ⁽²⁾ (으)로 중복 등록된다.

1-12 기본

최신순 정렬은 `heapq.`⁽¹⁾`(limit, stream, key=...)` 로 구현했다. 이때 메모리 사용량은 O(⁽²⁾) 로 고정된다.

1-13 심화

정렬 키는 같은 날짜의 동점을 가르기 위해 (⁽¹⁾, ⁽²⁾) 튜플로 둔다.

1-14 기본

데코레이터는 예외 처리·로그·시간 측정처럼 여러 기능에 흠여지는 ⁽¹⁾ 관심사를 분리하기 위한 장치다.

1-15 기본

`dataclasses.`⁽¹⁾`()` 는 기존 인스턴스에서 일부 필드만 바꾼 새 인스턴스를 만든다. `update` 의 부분 수정에 쓴다.

1-16 심화

`update` 에서 옵션을 아예 주지 않은 경우는 ⁽¹⁾, 빈 문자열로 준 경우는 ⁽²⁾ 로 구분된다.

1-17 기본

저장 파일은 최소 ⁽¹⁾ 개 이상으로 분리해야 하며, 기본 폴더는 `./data` 이되 ⁽²⁾ 옵션으로 변경 가능해야 한다.

1-18 기본

이 과제는 ⁽¹⁾ 라이브러리만 사용할 수 있고, `pip install` 이 필요한 패키지는 금지된다.

1-19 기본

`python -m budget_app` 이 동작하려면 패키지 안에 ⁽¹⁾ `.py` 파일이 있어야 한다.

1-20 심화

계층 분리가 지켜졌는지 판별하는 가장 단순한 기준: 서비스 계층에 `print` 와 ⁽¹⁾ _____ 호출이 한 군데도 없어야 한다.

1-21 기본

`itertools.` ⁽¹⁾ _____ 는 이터러블을 리스트로 바꾸지 않고 앞에서 N개만 잘라낸다.

1-22 심화

`id`는 `add` 마다 파일을 스캔해 최대 순번 ⁽¹⁾ _____ 로 계산한다. 이 방식의 알려진 한계는 마지막 거래를 지웠을 때 `id` 가 ⁽²⁾ _____ 된다는 점이다.

1-23 기본

`argparse`에서 여러 명령을 하나의 진입점 아래 두려면 ⁽¹⁾ _____ () 를 쓴다. `category add` 처럼 2단계로 중첩할 수도 있다.

1-24 심화

명세 7장 제약사항에 따라 옵션 표기는 ⁽¹⁾ _____ 대시로 통일한다. 단 `--help` 는 4장이 명시하므로 ⁽²⁾ _____ 표기를 함께 등록한다.

1-25 기본

예외를 잡아 사람이 읽을 메시지로 바꾸는 데코레이터는 ⁽¹⁾ _____ 계층에 건다. `print` 와 `sys.exit` 이 그 계층의 책임이기 때문이다.

1-26 심화

제너레이터를 반환하는 함수의 타입 힌트는 `->` ⁽¹⁾ _____ `[Transaction]` 형태로 쓴다. 리스트가 아님을 계약으로 표현하는 것이다.

1-27 기본

`category remove` 시 해당 카테고리를 쓰는 거래가 존재하면, 삭제를 ⁽¹⁾ _____ 거나 ⁽²⁾ _____ 을(를) 요구해야 한다.

1-28 기본

금액은 부동소수점 오차 때문에 `float` 이 아니라 ⁽¹⁾ _____ 로 다룬다. 명세도 '양수' ⁽²⁾ _____ '라고 규정한 다.

Part 2. 객관식 (5지선다)

가장 적절한 것을 하나 고르시오. 고른 이유도 여백에 한 줄로 적어두면 복습에 좋다.

2-01 심화

`list -limit 3` 을 `heapq.nlargest` 로 구현했다. 10만 줄짜리 파일에서 참인 것은?

- ① 파일에서 3줄만 읽고 즉시 멈춘다
- ② `sorted()` 로 전부 읽는 방식보다 실행 시간이 크게 줄어든다
- ③ 메모리 사용량이 약 3개 레코드 수준으로 고정된다
- ④ 파일을 두 번 읽어야 한다
- ⑤ 정렬 결과가 날짜순이 아니라 입력순이 된다

이유: _____

2-02 기본

`tags: list[str] = []` 로 `dataclass` 필드를 선언했을 때 실제로 생기는 문제는?

- ① 문법 오류가 발생한다
- ② 타입 힌트가 런타임에 강제되어 다른 타입을 못 넣는다
- ③ 모든 인스턴스가 같은 리스트 객체를 공유한다
- ④ JSON 직렬화가 불가능해진다
- ⑤ `dataclass`는 리스트 필드를 지원하지 않는다

이유: _____

2-03 기본

다음 중 저장소(storage) 계층이 **알아도 되는** 것은?

- ① 카테고리 등록되어 있어야 한다는 규칙
- ② 금액이 양수여야 한다는 규칙
- ③ 파일이 없으면 생성해야 한다는 사실
- ④ 예산 초과 시 경고를 출력해야 한다는 것
- ⑤ `argparse` 서브커맨드의 이름

이유: _____

2-04 기본

`os.replace()` 로 임시 파일을 교체하는 이유로 가장 정확한 것은?

- ① 파일 복사보다 속도가 빠르기 때문
- ② 같은 파일시스템 안에서 원자적이라 '반쯤 쓰인 파일'이라는 중간 상태가 존재하지 않기 때문
- ③ 파일 권한을 자동으로 맞춰주기 때문
- ④ 디스크 공간을 절약하기 때문
- ⑤ 다른 프로세스의 파일 잠금을 자동 해제하기 때문

이유: _____

2-05 기본

데코레이터에서 `functools.wraps` 를 생략하면 어떻게 되는가?

- ① 데코레이터가 아예 동작하지 않는다
- ② 감싼 함수의 `__name__` 이 `wrapper` 로 바뀌어 디버깅이 어려워진다
- ③ 예외가 바깥으로 전파되지 않는다
- ④ 원본 함수가 인자를 받지 못한다
- ⑤ 반환값이 항상 None이 된다

이유: _____

2-06 심화

`update` 를 옵션 기반으로 구현할 때, `-memo` 를 아예 주지 않은 경우와 `-memo ""` 로 빈 문자열을 준 경우를 구분하려면?

- ① `if args.memo:` 로 검사한다
- ② `if args.memo != "":` 로 검사한다
- ③ `default=None` 으로 두고 `if args.memo is not None:` 으로 검사한다
- ④ `nargs="?"` 를 주면 `argparse` 가 자동으로 구분해준다
- ⑤ `argparse` 만으로는 구분할 수 없고 별도 sentinel 클래스가 필요하다

이유: _____

2-07 심화

초기 카테고리 정책을 '파일이 비어있으면 시드'로 구현했다. 어떤 문제가 생기는가?

- ① 최초 실행이 실패한다
- ② 사용자가 카테고리를 전부 삭제한 뒤 재실행하면 기본값이 되살아난다
- ③ `import` 가 항상 실패한다
- ④ 거래 id가 중복 발급된다
- ⑤ `categories` 파일이 손상된다

이유: _____

2-08 심화

`-from` 옵션 구현에 대한 설명으로 옳은 것은?

- ① `add_argument("-from")` 후 `args.from` 으로 값을 읽으면 된다
- ② `add_argument("-from", dest="date_from")` 후 `args.date_from` 으로 읽어야 한다
- ③ 예약어는 옵션 이름으로 쓸 수 없어 `argparse`가 등록 시점에 에러를 낸다
- ④ `--from` 처럼 이중 대시로 쓰면 `args.from` 접근이 가능해진다
- ⑤ `from_` 처럼 언더스코어를 붙여 옵션 이름 자체를 바꿔야 한다

이유: _____

2-09 기본

`import` 결과가 `imported=5, skipped=1` 형태로 출력된다는 것이 함의하는 설계는?

- ① CSV 파일 5개를 읽었다
- ② 잘못된 행이 있어도 전체 작업이 중단되지 않고 행 단위로 격리된다
- ③ id가 중복된 행을 건너뛰었다
- ④ 실패한 요청을 5회 재시도했다
- ⑤ CSV 헤더 행을 건너뛰었다는 뜻이다

이유: _____

2-10 기본

`export` 명령의 조건 요구사항으로 옳은 것은?

- ① `-out` 만 주면 전체 거래를 내보낸다
- ② `-month` 또는 `-from / -to` 중 하나 이상을 반드시 받아야 한다
- ③ `-month` 가 항상 필수다
- ④ 아무 조건도 주지 않으면 전체 내보내기가 기본 동작이다
- ⑤ `-category` 가 필수다

이유: _____

2-11 심화

`@handle_errors` 데코레이터를 서비스 계층 함수에 걸고, 그 안에서 `print` 와 `sys.exit` 을 호출했다. 가장 심각한 결과는?

- ① 데코레이터가 동작하지 않는다
- ② 서비스가 CLI의 책임을 갖게 되어 계층이 무너지고, 서비스를 단독으로 호출·테스트할 수 없게 된다
- ③ exit code가 항상 0이 된다
- ④ 같은 예외가 두 번 발생한다
- ⑤ `functools.wraps` 를 쓸 수 없게 된다

이유: _____

2-12 기본

파일을 한 줄씩 흘려보내는 함수의 반환 타입 힌트로 가장 적절한 것은?

- ① `-> list[Transaction]`
- ② `-> Transaction`
- ③ `-> Iterator[Transaction]`
- ④ `-> generator`
- ⑤ `-> yield Transaction`

이유: _____

2-13 심화

id를 '매 add마다 파일을 스캔해 max+1' 로 발급할 때 실제로 발생하는 문제는?

- ① 마지막 거래를 삭제한 뒤 add하면 같은 id가 재사용된다
- ② 파일이 커지면 id가 음수로 오버플로된다
- ③ id가 문자열이라 정렬이 불가능해진다
- ④ 카운터 파일이 손상될 수 있다
- ⑤ 발생하는 문제가 없다

이유: _____

2-14 기본

금액을 `float` 으로 다루면 안 되는 이유로 가장 정확한 것은?

- ① float은 JSON에 저장할 수 없기 때문
- ② 부동소수점 오차 때문에 `0.1 + 0.2 != 0.3` 이 되어 합계가 어긋나기 때문
- ③ float에는 타입 힌트를 붙일 수 없기 때문
- ④ dataclass가 float 필드를 지원하지 않기 때문
- ⑤ float으로는 음수를 표현할 수 없기 때문

이유: _____

2-15 심화

`datetime.strptime("2024-1-5", "%Y-%m-%d")` 의 결과는?

- ① ValueError가 발생한다
- ② 정상 파싱되어 2024-01-05 가 된다
- ③ None을 반환한다
- ④ 2024-10-05 로 잘못 해석된다
- ⑤ TypeError가 발생한다

이유: _____

2-16 심화

`-help` 를 `argparse`에서 동작하게 하려면?

- ① 기본으로 자동 생성되므로 아무것도 안 해도 된다
- ② `add_help=False` 로 자동 생성을 끄고 `add_argument("-help", "--help", action="help")` 를 직접 등록한다
- ③ `prefix_chars` 를 바꿔야 한다
- ④ `argparse`로는 불가능하다
- ⑤ 자동 생성된 `-h` 의 이름을 `rename`하면 된다

이유: _____

2-17 기본

명세가 요구하는 오류 출력 형태는?

- ① 스택트레이스 전체를 그대로 출력
- ② 오류 코드 번호만 출력
- ③ 원인 + 해결 힌트를 사람이 읽을 문장으로 출력
- ④ 로그 파일에만 기록하고 화면에는 출력하지 않음
- ⑤ 아무것도 출력하지 않고 조용히 종료

이유: _____

2-18 심화

다음 중 '계충이 썼다'고 볼 수 있는 신호가 **아닌** 것은?

- ① `services.py` 안에 `input()` 호출이 있다
- ② `storage.py` 가 '등록된 카테고리인지' 검사한다
- ③ `cli.py` 가 `Path(data_dir) / "transactions.jsonl"` 을 직접 조립한다
- ④ `models.py` 가 `dataclass` 정의만 하고 파일·출력 관련 모듈을 전혀 `import`하지 않는다
- ⑤ `services.py` 가 `sys.exit(1)` 을 호출한다

이유: _____

2-19 기본

`category remove` 시 해당 카테고리를 사용하는 거래가 존재하면 어떻게 해야 하는가?

- ① 무시하고 카테고리만 삭제한다
- ② 해당 카테고리를 쓰는 거래도 함께 삭제한다
- ③ 삭제를 막거나, 대체 카테고리를 요구한다
- ④ 카테고리를 '삭제됨' 상태로 표시만 한다
- ⑤ 명세에 규정이 없으므로 자유롭게 정한다

이유: _____

2-20 심화

`summary` 에서 예산 사용률(%)을 계산할 때 반드시 방어해야 하는 것은?

- ① 예산이 0이거나 설정되지 않은 달의 나눗셈
- ② 수입이 지출보다 큰 경우
- ③ 카테고리가 10개를 넘는 경우
- ④ 월이 12월인 경우
- ⑤ 태그가 하나도 없는 경우

이유: _____

2-21 심화

`add`, `update`, `import` 세 경로가 모두 같은 검증 로직을 타야 하는 가장 큰 이유는?

- ① 코드 줄 수를 줄이기 위해
- ② 한 경로에만 검증이 빠지면 그 경로로 규칙을 어긴 데이터가 저장 파일에 들어가기 때문
- ③ 타입 힌트를 통일하기 위해
- ④ `argparse` 옵션 이름을 맞추기 위해
- ⑤ 테스트 파일을 하나로 합치기 위해

이유: _____

2-22 기본

`search` 에서 6개 필터(`-from -to -category -type -q -tag`)를 조합할 때 가장 좋은 구조는?

- ① 조건마다 if문을 중첩해 6단계로 쌓는다
- ② 조건 조합마다 별도 함수를 만든다($2^6 = 64$ 개)
- ③ 주어진 조건만 술어 함수로 만들어 리스트에 모으고 `all(f(tx) for f in filters)` 로 평가한다
- ④ 전체를 리스트로 읽은 뒤 pandas로 필터링한다
- ⑤ SQL 문자열을 만들어 sqlite3에 넘긴다

이유: _____

Part 3. 코드 빈칸 채우기

빈칸에 들어갈 코드를 쓰시오. 아래 여백에 이유를 함께 적을 것.

3-01 심화 원자적 재작성 (update / delete 공용)

```
import os, json, tempfile

def rewrite_atomic(path: str, records: list[dict]) -> None:
    directory = os.path.dirname(path)
    fd, tmp = tempfile.mkstemp(dir=(1)_____)
    with os.fdopen(fd, "w", encoding="utf-8") as f:
        for r in records:
            f.write(json.dumps(r, ensure_ascii=(2)_____) + "\n")
    os.(3)_____(tmp, path)
```

3-02 기본 제너레이터 기반 스트리밍 읽기

```
from collections.abc import Iterator

def stream_transactions(path: str) -> (1)_____[Transaction]:
    with open(path, encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line:
                continue
            (2)_____ Transaction(**json.loads(line))
```

3-03 기본 예외 처리 데코레이터 (CLI 진입점에 적용)

```
import sys, functools

def handle_errors(func):
    @functools.(1) (func)
    def wrapper(*args, **kwargs):
        try:
            return func(*args, **kwargs)
        except ValidationError as e:
            print(f"[오류] {e}")
            print(f"[힌트] {e.hint}")
            sys.exit((2) )
    return wrapper
```

3-04 심화 옵션 기반 부분 수정

```
import dataclasses

def apply_update(tx: Transaction, amount=None, memo=None) -> Transaction:
    changes = {}
    if amount is not (1) :
        changes["amount"] = amount
    if memo is not None:
        changes["memo"] = memo
    return dataclasses.(2) (tx, **changes)
```

3-05 심화 최신순 상위 N건 (메모리 O(limit))

```
import heapq

latest = heapq.(1) (
    limit,
    stream_transactions(path),
    key=lambda tx: (tx.(2), tx.(3)),
)
```

Part 4. 서술형

문장으로 설명하시오. 과제 목표는 "스스로 설명할 수 있는 것"이므로 키워드 나열이 아니라 완성된 문장으로 쓸 것.

4-01 심화

파일 기반 저장에서 `update / delete` 가 `add` 보다 훨씬 위험한 이유를 설명하고, 그 위험을 줄이기 위해 채택한 방법을 단계별로 서술하시오.

4-02 기본

모델 / 저장소 / 서비스 / CLI 각 계층이 **몰라야 하는 것**을 하나씩 쓰고, 그렇게 나누면 구체적으로 무엇이 쉬워지는지 서술하시오.

4-03 기본

yield 기반 제너레이터가 리스트를 반환하는 함수와 어떻게 다른지 설명하고, 이 과제에서 왜 필요한지 서술하시오.

4-04 심화

데코레이터로 예외 처리를 분리해 얻는 것을 쓰고, 그 데코레이터를 서비스가 아니라 **CLI 진입점**에 건 이유를 서술하시오.

4-05 기본

타입 힌트가 이 과제에서 실제로 도움이 된 지점을 **구체적인 코드 예시**와 함께 서술하시오. (힌트: `Iterator[Transaction]` 과 `list[Transaction]` 의 차이)

4-06 심화

명세는 "파일 전체를 한 번에 로드하지 말라"와 "최신순으로 출력하라"를 동시에 요구한다. 이 둘이 왜 원리적으로 충돌하는지 설명하고, 채택한 해법과 그 **해법이 포기한 것**을 서술하시오.

4-07 심화

id 발급을 '파일 스캔 max+1' 로 정했다. '별도 카운터 파일' 방식과 비교해 장점 2가지와 단점 2가지를 서술하시오.

4-08 심화

import/export CSV 스키마에 `id` 컬럼이 없다. 이로 인해 export → import 왕복 시 무슨 일이 생기는가? 이것을 '버그'가 아니라 '명세대로의 동작'이라고 볼 수 있는 근거는 무엇인가?

4-09 심화

`add` , `update` , `import` 세 경로가 모두 같은 검증 로직을 타야 하는 이유를 쓰고, 타지 않을 경우 생기는 구멍을 구체적인 시나리오로 서술하시오.

4-10 심화

명세는 "스택트레이스 출력 금지"를 요구한다. 그런데 모든 예외를 무조건 잡아 삼키면 어떤 새로운 문제가 생기는가? 내가 정의한 예외와 예상하지 못한 버그를 각각 어떻게 다르게 다룰 것인지 서술하시오.

4-11 심화

`category remove` 의 '사용 중인지' 검사 로직은 어느 계층에 두어야 하는가? 그 검사가 transactions 파일 전체 스캔을 필요로 한다는 점을 고려해 근거와 함께 서술하시오.

4-12 심화

이 프로그램에 보너스 과제인 '월별 반복 내역(월급·월세)' 기능을 추가한다고 하자. 4계층 각각에서 무엇을 바꿔야 하는가? 새 저장 파일이 필요한가? 그 이유는?

Part 5. 코드 진단 (종합)

실제로 나올 법한 잘못된 코드다. 무엇이 왜 잘못됐고 어떻게 고칠지 쓰시오.

5-01 심화

다음 코드에서 **계층 원칙 위반**을 찾아 지적하고, 어떻게 고쳐야 하는지 쓰시오.

```
# services.py
def add_transaction(date, type_, category, amount):
    if not is_valid_date(date):
        print("[오류] 날짜 형식이 올바르지 않습니다 (YYYY-MM-DD).")
        sys.exit(1)
    if category not in load_categories():
        print("[오류] 등록되지 않은 카테고리입니다.")
        sys.exit(1)
    ...
```

5-02 심화

다음 `list` 구현은 명세를 두 가지 위반한다. 각각 지적하고 올바른 구현 방향을 쓰시오.

```
def list_transactions(path, limit=10):  
    records = [json.loads(l) for l in open(path)]  
    records.sort(key=lambda r: r["date"], reverse=True)  
    return records[:limit]
```

5-03 심화

다음 카테고리 로딩 코드에는 우리가 내린 **설계 결정 2번**을 위반하는 버그가 있다. 찾아서 고치시오.

```
def load_categories(path: Path) -> list[str]:
    if path.exists():
        cats = read_lines(path)
        if not cats:
            cats = DEFAULT_CATEGORIES
            write_all(path, cats)
        return cats
    write_all(path, DEFAULT_CATEGORIES)
    return DEFAULT_CATEGORIES
```

정답 및 해설

서술형·코드진단은 정답이 하나가 아니다. 아래는 **채점 포인트**이므로, 본인 답안에 이 요소들이 들어 있는지로 확인할 것.

1-01 (1) JSONL

1-02 (1) 공유 / (2) default_factory

1-03 (1) 일시정지(중단)

1-04 (1) replace / (2) 원자성

1-05 (1) wraps

1-06 (1) 0 / (2) 0

1-07 (1) 예약어(키워드) / (2) dest

1-08 (1) 없을 / (2) 비었음

1-09 (1) ""(빈 문자열) / (2) 빈 줄

1-10 (1) False

1-11 (1) id / (2) 새 id

1-12 (1) nlargest / (2) limit

1-13 (1) date / (2) id

1-14 (1) 횡단(cross-cutting)

1-15 (1) replace

1-16 (1) None / (2) ""

1-17 (1) 3 / (2) -data-dir

1-18 (1) 표준

1-19 (1) __main__

1-20 (1) sys.exit

1-21 (1) islice

1-22 (1) +1 / (2) 재사용

1-23 (1) add_subparsers

1-24 (1) 단일(-) / (2) 양쪽(-help / --help)

1-25 (1) CLI

1-26 (1) Iterator

1-27 (1) 막 / (2) 대체 카테고리

1-28 (1) int(정수) / (2) 정수

2-01 3번

nlargest는 파일 끝까지 읽으므로 I/O(=시간)는 줄지 않는다. 힙에 limit개만 유지하므로 이점은 메모리다. 그래서 시연도 실행 시간이 아니라 tracemalloc 등 메모리 관점으로 증명해야 한다.

2-02 3번

가변 기본값은 클래스 정의 시점에 한 번만 만들어져 모든 인스턴스가 공유한다. `field(default_factory=list)` 를 써야 인스턴스마다 새 리스트가 생긴다. (dataclass는 이 실수를 감지해 에러를 내주기도 하지만, 원리를 아는 게 핵심)

2-03 3번

1번2번4번은 업무 규칙 → 서비스. 5번은 CLI. 3번은 '파일'에 관한 사실이므로 저장소의 책임이다.

2-04 2번

속도가 아니라 원자성이 목적이다. 원본에 직접 덮어쓰다가 중간에 죽으면 데이터가 날아가지만, 임시 파일에 완전히 쓴 뒤 교체하면 성공 아니면 원본 유지 둘 중 하나만 남는다.

2-05 2번

동작 자체는 한다. 다만 이름·docstring 같은 메타데이터가 사라져서 스택이나 로그에 전부 `wrapper` 로 찍힌다.

2-06 3번

1번은 빈 문자열도 falsy라 둘을 구분 못 한다(메모를 지우려는 의도가 무시됨). 2번은 미지정(None)일 때도 참이 되어 None을 그대로 덮어쓴다. `is not None` 만이 '안 준 것'과 '빈 값으로 준 것'을 정확히 가른다.

2-07 2번

사용자가 의도적으로 지운 것을 프로그램이 되살리는 셈이 된다. 그래서 '파일이 없을 때(=최초 생성 시점)만' 으로 조건을 좁혀야 한다. 한 단어 차이가 버그를 만든다.

2-08 2번

argparse는 `-from` 등록을 정상 처리한다(3번 오답). 문제는 파이썬 쪽이다 — `args.from` 은 `from` 이 예약어라 문법 오류다(1번4번 오답). `dest=` 로 속성 이름만 바꿔주면 된다. 옵션 이름은 명세대로 `-from` 을 유지해야 하므로 5번도 오답.

2-09 2번

`skipped` 라는 개념이 존재한다는 건 '한 행의 실패가 전체를 죽이지 않는다'는 뜻이다. 행마다 try/except로 감싸고 카운터만 올리는 구조가 필요하다.

2-10 2번

명세: "export는 -month YYYY-MM 또는 -from/-to 중 하나 이상 조건을 필수로 받는다." 무조건 전체를 내보내는 동작은 허용되지 않는다.

2-11 2번

동작은 한다. 문제는 구조다. 서비스가 `sys.exit` 을 부르면 REPL이나 테스트에서 호출하는 순간 프로세스가 죽어버려서, 과제 목표인 '계층별 책임 설명'이 성립하지 않는다.

2-12 3번

1번로 쓰면 '전부 메모리에 올린다'는 잘못된 계약을 선언하는 셈이다. 타입 힌트의 가치는 이런 **계약의 명시**에 있다.

2-13 1번

`TX-000012` 를 지우면 최대 순번이 11로 내려가므로, 다음 add가 다시 `TX-000012` 를 발급한다. 예전에 export한 CSV와 의미가 어긋날 수 있어 README에 명시해야 한다. 4번은 카운터 파일 방식(안 B)의 단점이지 이 방식의 단점이 아니다.

2-14 2번

명세도 '양수 정수'로 못 박았다. summary의 합계·잔액이 미세하게 어긋나는 버그를 원천 차단한다.

2-15 2번

strptime은 zero-padding에 **관대하다**. 즉 `strptime` 통과 = 형식이 정확하다 가 아니다. `YYYY-MM-DD` 형식을 엄격히 강제하려면 정규식이나 길이 검사를 따로 해야 한다.

2-16 2번

argparse가 자동으로 붙여주는 건 `-h` 와 `--help` 뿐이다. `-help` 는 직접 등록해야 한다. 서브커맨드에도 전부 필요하므로 부모 파서(`parents=[...]`)로 한 번만 정의하는 게 맞다.

2-17 3번

명세: "오류는 스택트레이스 대신 원인 + 해결 힌트로 출력한다." 그리고 종료 코드는 0이 아니어야 한다.

2-18 4번

4번은 오히려 계층이 **잘 지켜진** 모습이다. 3번이 위반인 이유: 파일 경로 조립은 저장소의 책임이고, CLI가 알면 저장 방식이 바뀔 때 CLI까지 고쳐야 한다.

2-19 3번

명세: "삭제를 막거나 / 대체 카테고리를 요구해야 합니다." 이 검사를 하려면 transactions 전체를 스캔해야 하므로, 어느 계층에 돌지도 함께 결정해야 한다.

2-20 1번

`지출 / 예산 * 100` 에서 예산이 0이면 ZeroDivisionError가 난다. 예산 미설정 달은 애초에 사용률 줄을 출력하지 않는 게 맞다 — 이것도 설계 결정이다.

2-21 2번

예: import에만 카테고리 검증이 빠지면, CSV로는 미등록 카테고리가 들어가고 그 뒤 `summary` 의 카테고리별 집계에 유령 카테고리가 나타난다. 중복 제거는 부수 효과일 뿐, 본질은 **규칙의 단일 출처**다.

2-22 3번

4번은 외부 라이브러리 금지 위반. 5번은 저장 방식 자체를 바꾸는 것이라 명세 위반. 3번은 조건이 늘어도 리스트에 하나 추가하면 끝이고 제너레이터 파이프라인도 유지된다.

3-01 (1) directory (원본과 같은 디렉토리) / (2) False / (3) replace

(1) 임시 파일이 **다른 파일시스템**에 있으면 `os.replace`가 원자적이지 않다. 반드시 대상과 같은 디렉토리에 만든다. (3) `os.rename` 은 윈도우에서 대상이 존재하면 실패한다. `os.replace` 가 플랫폼 무관하게 덮어쓴다.

3-02 (1) Iterator / (2) yield

`for line in f:` 자체가 이미 한 줄씩 읽는 지연 평가다. 거기에 `yield` 를 얹으면 호출자가 요청할 때만 다음 줄을 읽는 파이 프라인이 완성된다.

3-03 (1) wraps / (2) 1 (0이 아닌 값)

이 데코레이터가 CLI 핸들러에 붙기 때문에 서비스 계층은 `print` / `sys.exit` 을 끝까지 모른 채 예외를 `raise` 만 하면 된다.

3-04 (1) None / (2) replace

`if amount:` 로 쓰면 `-amount 0` 이 무시된다(0은 falsy). `is not None` 이라야 '안 준 것'과 '0/빈 문자열로 준 것'이 갈린다. 다만 금액은 양수 검증에서 따로 걸러야 한다.

3-05 (1) nlargest / (2) date / (3) id

정렬 키를 튜플로 두면 날짜가 같을 때 id로 안정적으로 갈린다. `nlargest` 는 내부적으로 크기 `limit` 의 힙만 유지하므로 파일이 아무리 커도 메모리는 고정이다.

4-01 — (서술형)

add는 append 한 줄이라 실패해도 기존 데이터가 멀쩡하다. update/delete는 파일 전체 재작성이라 쓰는 도중 중단되면 원본이 절반만 남는다 → 임시 파일에 전량 기록 → `os.replace`로 원자적 교체.

4-02 — (서술형)

모델: 파일·출력·업무규칙 / 저장소: 업무규칙 / 서비스: `argparse·input·print` / CLI: 파일 경로·JSON. 이점: 서비스를 REPL에서 단독 호출 가능, 저장소를 임시 디렉토리로 단독 테스트 가능, 저장 포맷 교체 시 저장소만 수정.

4-03 — (서술형)

리스트는 호출 즉시 전부 계산·적재. 제너레이터는 요청 시점에 한 개씩 계산하고 그 자리에서 일시정지. 명세가 '파일 전체를 한 번에 로드하지 말 것'을 명시적으로 요구.

4-04 — (서술형)

10개 명령의 동일한 try/except·종료 코드 처리를 한 곳으로 모음. CLI에 거는 이유: `print`와 `sys.exit`은 CLI의 책임이고, 서비스에 걸면 서비스가 프로세스를 죽이게 되어 단독 호출·테스트가 불가능해짐.

4-05 — (서술형)

반환 타입이 계약을 선언한다. Iterator로 쓰면 '전부 메모리에 올리지 않는다'가 시그니처만 보고 드러나고, list로 잘못 바꾸면 스트리밍 요구 위반이 시그니처에서 바로 보인다.

4-06 — (서술형)

정렬은 마지막 줄까지 봐야 순서를 확정할 수 있으므로 '앞에서 N개만 읽고 멈추기'와 양립 불가. 채택: `heapq.nlargest` — 메모리는 $O(\text{limit})$ 로 고정되나 I/O는 전체를 읽는다(=시간 이점은 포기). 포기하지 않은 것: 날짜순 정확성.

4-07 — (서술형)

장점: (1) 카운터가 데이터에서 파생돼 불일치가 구조적으로 불가능 (2) 저장 파일 3개 유지, 원자성 고민 불필요. 단점: (1) add가 $O(n)$ (2) 마지막 거래 삭제 시 id 재사용.

4-08 — (서술형)

왕복하면 동일 내용이 새 id로 중복 등록된다. 근거: 명세가 CSV 최소 스키마를 `date/type/category/amount/memo/tags` 6개로 고정했고 id를 포함시키지 않았다. 따라서 import는 언제나 신규 등록 경로다 → README에 명시할 것.

4-09 — (서술형)

규칙의 단일 출처. 예: import에만 카테고리 검증이 빠지면 CSV로 미등록 카테고리가 유입되고, 이후 summary의 카테고리별 집계에 유령 카테고리가 나타나며, category remove의 '사용 중' 검사도 어긋난다.

4-10 — (서술형)

전부 삼키면 진짜 버그(오타로 인한 AttributeError 등)까지 '사용자 오류'처럼 보이게 되어 디버깅이 불가능해진다. 구분: 커스텀 예외 (ValidationError 등)는 원인+힌트로 번역, 그 외는 최소한 예외 종류와 위치를 남기거나 디버깅 플래그에서만 상세 출력.

4-11 — (서술형)

서비스 계층. '카테고리를 지우려면 사용 중이면 안 된다'는 업무 규칙이기 때문. 저장소는 '거래를 스트리밍한다', '카테고리를 재작성한다'는 능력만 제공하고, 두 저장소를 조합해 판단하는 건 서비스의 일.

4-12 — (서술형)

모델: Recurring dataclass 추가. 저장소: recurrings 파일 스트림/재작성(→ 4번째 파일 필요, 규칙 자체를 영속해야 하므로). 서비스: 특정 월에 규칙을 전개해 Transaction 생성 + 중복 생성 방지 규칙. CLI: recurring 서버커맨드.

5-01 — (서술형)

서비스가 print와 sys.exit을 호출한다 → CLI의 책임 침범. 서비스는 `raise ValidationError("날짜 형식이 올바르지 않습니다", hint="예: 2024-01-15")` 만 하고, CLI 진입점의 `@handle_errors`가 잡아서 출력·종료하도록 고친다. 이렇게 해야 서비스를 REPL/테스트에서 단독 호출할 수 있다.

5-02 — (서술형)

(1) 리스트 컴프리헨션으로 파일 전체를 한 번에 메모리에 로드 — '한 번에 로드하지 말 것' 위반. (2) 제너레이터를 전혀 쓰지 않음 — '제너레이터 기반 스트리밍' 위반. 추가로 파일 핸들을 닫지 않는다(with 미사용). → `stream_transactions()` 제너레이터 + `heapq.nlargest(limit, ..., key=(date, id))` 로 교체.

5-03 — (서술형)

`if not cats:` 블록이 '파일이 비었으면 시드'를 수행한다 → 사용자가 카테고리를 전부 삭제하고 재실행하면 기본값이 부활한다. 결정 2번은 '파일이 없을 때만 시드'다. → `if not cats:` 블록을 통째로 제거하고, 파일이 없는 경로(else)에서만 시드하도록 남긴다.
